门与模型一起训练：门的稀疏性是结构选择

> 何淇 · 与 DeepSeek（AI）协作

> 数字生命系列 · 执行器工程（汇总篇，取代论文-44 的联合训练与执行部分）

> 日期：2026-11-13

> 状态：草稿。数据来自论文-44（DOI 10.5281/zenodo.23858151）及工程笔记 01/02；

> 本篇聚焦"训练方式"与"执行方式"两个变量，不重复 44 的细节。

一、问题

上一篇把门定成"重配比"：门调的是已有通路的参与度，不增加方向，所以它跨不过维度门槛。

但重配比有一个前提没问：**这些通路是什么时候形成的？**

如果门在模型训练的一开始就在，那它参与塑造通路——模型从一开始就知道"有些层可能被跳过"，会把计算安排到别处。如果门是模型冻结之后才加的，那它只能在一个已经定型的结构上做取舍，而那些层在预训练时都被"用惯了"。

同样的门，同样的参数，训练方式不同，它能做到的事可能完全不同。这篇只测这个变量，再补上执行侧（怎么算才算省）。

二、设置

门的形式：每层一个内容门 σ(W·h+b)，每个 token 每层各自决策。训练时用拉格朗日乘子控制平均活跃层数：损失 = 语言建模损失 + λ·平均活跃层数，λ 自适应调整。不人工指定哪层该关。

两种执行方式：

- 保留 K/V 的稀疏执行：K/V 全算，门关的 token 跳过 Q/注意力输出/MLP；

- 容量路由（MoD 风格）：每层固定容量 K，按门分数取 top-K 个 token。

对照：从零训练（1.2M 字符模型、GPT-2 124M、Qwen2.5-1.5B）vs 预训练 + LoRA 后训练（Qwen2.5-3B/7B）。数据为 tinyshakespeare（1MB）或 TinyStories（20MB/100MB），评测为验证集 PPL。

三、结果：训练方式决定门能做到什么

| 设置 | 平均用层 | 代价（PPL） |

|---|---|---|

| 1.2M 字符模型，从零 | 2.97/6（50%） | ≈0 |

| GPT-2 124M，从零，1MB | 5.30/12（44%） | +6.8% |

| GPT-2 124M，从零，100MB | 5.81/12（48%） | +2.7% |

| Qwen2.5-1.5B，从零，100MB | 12.32/28（44%） | +1.1% |

| Qwen2.5-3B，预训练+LoRA | 32.79/36（91%） | +18% |

| Qwen2.5-7B，预训练+LoRA | 25.70/28（92%） | +38% |

| Qwen2.5-7B，目标 24 层 | 24.01/28（86%） | +85% |

| Qwen2.5-7B，目标 22 层 | 22.27/28（80%） | +82% |

三个观察：

一是从零训练时，门可以便宜到几乎免费：1.2M 省一半层数不损质量；1.5B 省 56% 层数只多 1.1%。

二是从零训练时，门自己会做结构选择。1.5B 上最后 9 层（L19~L27）几乎全部关闭，前面保留大部分层；GPT-2 上是第 3 层几乎不用、末两层回到 0.74~0.75。这不是我们指定的，是训练自己找到的。

三是冻结 + 轻量后训练时，门压不动。3B/7B 的层数只压到 91%~92%，代价 +18%~+38%；想压到 86%/80%，代价跳到 +85%/82%，7B 那次还发生了门控塌缩（PPL 升到 1243，层数掉到 20）。这不是"大模型不能压缩"的结论——只能说明这种后训练方式不行。

四、门本身不伤质量，伤在"怎么执行"

在 GPT-2 门控模型（7.48/12 层，PPL 12.03）上：

- 只算门开 token、K/V 全留（open_only）：PPL 12.032，与稠密门控 12.033 精确等价；

- 容量路由（门开优先 + 静态 K_max，factor=1.00）：名义省 37.9% 计算，PPL 12.107（+0.6%）；

- 质量拐点在 K_max ≈ 平均门开数（factor 1.0~1.05）；factor=0.95 时 PPL 14.458。

这说明门控本身不带来质量损失；差异来自执行方式。代价出现在更强的地方：注意力子集化（跳过 token 不产生 K/V）在已训练模型上 +6.8%（quantized）、half +16.6%、quarter +55%；从零训练两次都崩（PPL 350.99 / 393.71）。这不是"子集化不可行"的结论——MoD 原方法用独立路由器 + 容量路由，在更大规模上验证过这个语义；我们的路由方式不够。

[事实] 真实墙钟（重要负结果）：自写 gather/索引实现下，batch 32 为 0.89~1.22x，batch 64 为 1.01x；早期一次 1.22x 与后期 0.89x 的差异说明结果高度依赖实现细节。我们测到的是名义计算量，真实加速是独立的工程问题。

分支独立门控失败：每层两个门（注意力、MLP 分开），从零训练 12000 步 final PPL 14.484；联合门控 4000 步就到 12.033。分支拆分评测：MLP 压到 25% → PPL 558.95；注意力压到 25% → 69.45。初步图景：MLP（权重承载的变换）与 K/V（记忆）是承重部分，Q 侧更新相对可省——单模型观察，不能当结论。

五、解释与边界

[解释] 从零训练时，全网在"可能跳层"的约束下共同学习，可以把计算集中到少数层；冻结模型没有这个机会：每一层都被预训练用惯了，LoRA 的低秩更新改不动这个结构。所以门的稀疏性更像一个**结构选择**，而不是事后能强加的适配。

边界：单种子为主；数据 1MB~100MB；仅 PPL；从零训练组欠训练（8000 步约 0.33 epoch），绝对数不代表真实能力；未与 MoD 原文做同设置对比；未测 1B 以上从零训练；墙钟结果受实现影响大。

六、与已有工作的关系

门控、稀疏执行、容量路由、早退同属条件计算；MoD 已报告过容量路由的可行性，LayerDrop 等也做过训练时的层丢弃。我们不声称提出新方法。

我们的观察是：**同一个门，从零训练与冻结后加，能做到的事差一个量级**；而且从零训练时门会自己关闭整段末端层，这像是把"哪些层是承重的"学了出来。这个对照在已有文献里不常被单独强调，但它是一个工程事实，不是方法贡献。

七、和第一篇的关系

第一篇说门是重配比，跨不过维度门槛；这篇补上前提：重配比发生在什么结构上，取决于门是否参与训练。冻结时，门只能在定型结构里选；共同训练时，门参与塑造那个结构，于是它能"关掉一半层"而不伤功能。

一句话：**门在冻结模型里是开关，在共同训练里是结构选择。**